Teoria Współbieżności Zadanie domowe Lab 7

Oliwia Dyszewska

grudzień 2025

1 Opis zadania

Zadanie polega na wykonaniu następujących etapów (dla macierzy o rozmiarze N): (Proszę skoncentrować się na przykładzie z wykładu.)

- Proszę zlokalizować niepodzielne czynności wykonywane przez algorytm, nazwać je oraz zbudować alfabet w sensie teorii śladów
- Proszę skonstruować relację zależności dla alfabetu, opisującego algorytm eliminacji Gaussa-Jordana
- Proszę przedstawić algorytm eliminacji Gaussa-Jordana w postaci ciągu symboli alfabetu.
- Proszę wygenerować graf zależności Diekerta.
- Proszę przekształcić ciąg symboli opisujący algorytm do postaci normalnej Foaty.

Proszę zaprojektować i zaimplementować współbieżny algorytm eliminacji Gaussa-Jordana. W szczególności proszę zwrócić uwagę na implementację jak najlepiej odwzorowującą graf zależności lub postać normalną Foaty. Sugerowane rozwiązania to wykorzystanie biblioteki GALOIS, biblioteki CILK, CUDA lub odpowiednich języków programowania. Program ma działać dla zadanych rozmiarów macierzy N

2 Teoria

2.1 Niepodzielne czynności oraz alfabet w sensie teorii śladów

Poniżej znajduje się poglądowy rysunek $Mx = y$, gdzie M jest macierzą kwadratową, natomiast x oraz y są wektorami. Każdy element macierzy indeksowany jest jako $M_{i,j}$, a wektora y jako y_i.

$$\begin{bmatrix} M_{1,1} & M_{1,2} & M_{1,3} \\ M_{2,1} & M_{2,2} & M_{2,3} \\ M_{3,1} & M_{3,2} & M_{3,3} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

Dla uproszczenia zapisu zastosujemy poniższy zapis.

$$\left[\begin{array}{ccc|c} M_{1,1} & M_{1,2} & M_{1,3} & M_{1,4} \\ M_{2,1} & M_{2,2} & M_{2,3} & M_{2,4} \\ M_{3,1} & M_{3,2} & M_{3,3} & M_{3,4} \end{array}\right]$$

Wyróżniamy następujące rodzaje operacji na elementach macierzy;

- $A_{k,i}$ - *dzielenie:*
$$m_{k,i} \leftarrow \frac{M_{k,i}}{M_{i,i}}$$
- $B_{k,j,i}$ - *mnożenie:*
$$n_{k,j,i} \leftarrow M_{i,j} * m_{k,i}$$
- $C_{k,j,i}$ - *odjęcie:*
$$M_{k,j} \leftarrow M_{k,j} - n_{k,j,i}$$

Alfabet w sensie teorii śladów to (zakładamy, że n to rozmiar macierzy):

$$\Sigma = \{\, A_{k,i},\, B_{k,j,i},\, C_{k,j,i} \mid 1 \leq i < n,\ i < k \leq n,\ i \leq j \leq n+1 \}\,.$$

Warunek $j \leq n+1$ wynika z pracy na macierzy rozszerzonej $[A \mid b]$ o wymiarze $n \times (n+1)$.

2.2 Relacja zależności i niezależności

Relacja zależności D

$$D_1 = \{(A_{k,i}, B_{k,j,i}) \mid A_{k,i}, B_{k,j,i} \in \Sigma\} \quad \text{(bo } B \text{ czyta } m_{k,i} \text{ zapisane przez } A\text{)},$$

$$D_2 = \{(B_{k,j,i}, C_{k,j,i}) \mid B_{k,j,i}, C_{k,j,i} \in \Sigma\} \quad \text{(bo } C \text{ czyta } n_{k,j,i} \text{ zapisane przez } B\text{)},$$

$$D_3 = \{(C_{k_c,j_c,i_c}, B_{k_b,j_b,i_b}) \mid B_{k_b,j_b,i_b}, C_{k_c,j_c,i_c} \in \Sigma \ \wedge \ k_c = i_b \ \wedge \ j_c = j_b\} \quad \text{(bo } C \text{ zapisuje } M_{i_b,j_b} \text{ czytane przez } B\text{)},$$

$$D_4 = \{(C_{k_c,j_c,i_c}, A_{k_a,i_a}) \mid A_{k_a,i_a}, C_{k_c,j_c,i_c} \in \Sigma \ \wedge \ (j_c = i_a \wedge (k_c = k_a \vee k_c = i_a))\}$$
$$\text{(bo } A \text{ czyta } M_{k_a,i_a} \text{ oraz } M_{i_a,i_a}\text{, które } C \text{ zapisuje)},$$

$$D_5 = \{(C_{k,j,i_1}, C_{k,j,i_2}) \mid C_{k,j,i_1}, C_{k,j,i_2} \in \Sigma \ \wedge \ i_1 \neq i_2\} \quad \text{(oba zapisują tę samą komórkę } M_{k,j}\text{)}.$$

Końcowo:

$$D = \mathrm{sym}\{\{D_1 \cup D_2 \cup D_3 \cup D_4 \cup D_5\}^+\} \ \cup \ I_\Sigma$$

Relacja niezależności I

$$I = \Sigma^2 - D$$

2.3 Algorytm eliminacji Gaussa w postaci ciągu symboli alfabetu

Skonstrujemy ciąg operacji odpowiadający eliminacji. Zdefiniujmy podciąg $p_{k,i}$, który odpowiada wyzerowaniu elementu $M_{k,i}$ (przez odjęcie odpowiedniej wielokrotności wiersza i od wiersza k):

$$p_{k,i} = \big(A_{k,i},\ (B_{k,i,i}, C_{k,i,i}),\ (B_{k,i+1,i}, C_{k,i+1,i}),\ \ldots,\ (B_{k,n+1,i}, C_{k,n+1,i})\big),$$

gdzie $A_{k,i}$ wyznacza mnożnik $m_{k,i}$, a następnie dla kolejnych kolumn j wykonujemy parę $(B_{k,j,i}, C_{k,j,i})$ aktualizującą $M_{k,j}$.

Cały algorytm eliminacji (bez podstawiania wstecz) wyraża się jako konkatenacja tych podciągów:

$$w_{\mathrm{E}} = \big(p_{2,1}, p_{3,1}, \ldots, p_{n,1},\ p_{3,2}, p_{4,2}, \ldots, p_{n,2},\ \ldots,\ p_{n,n-1}\big),$$

.

2.4 Graf zależności Diekerta

Wyznaczamy zbiór krawędzi K grafu zależności Diekerta dla słowa w_{E} opisującego eliminację (dla macierzy rozmiaru N). Ponieważ graf Diekerta jest z definicji konstruowany dla *konkretnego słowa*, na rysunku przedstawiamy przykład dla małego rozmiaru $N = 4$ (dla większych N wykres staje się mało czytelny).

Typy konfliktów (kandydaci na krawędzie). Najpierw opisujemy, które *typy operacji* mogą być w konflikcie (odczyt/zapis).

$$K_1 = \{(A_{k,i}, B_{k,j,i}) \mid A_{k,i}, B_{k,j,i} \in \Sigma\},$$
$$K_2 = \{(B_{k,j,i}, C_{k,j,i}) \mid B_{k,j,i}, C_{k,j,i} \in \Sigma\},$$
$$K_3 = \{(C_{k_c,j_c,i_c}, B_{k_b,j_b,i_b}) \mid C_{k_c,j_c,i_c}, B_{k_b,j_b,i_b} \in \Sigma \ \wedge \ k_c = i_b \ \wedge \ i_c = i_b - 1 \ \wedge \ j_c \neq i_b \wedge \ j_b \neq i_b \wedge \ j_c = j_b\},$$
$$K_4 = \{(C_{k_c,j,i_c}, A_{k_a,i_a}) \mid C_{k_c,j,i_c}, A_{k_a,i_a} \in \Sigma \ \wedge \ j = i_a \ \wedge \ i_c = i_a - 1 \ \wedge \ (k_c = k_a \ \vee \ k_c = i_a)\},$$
$$K_5 = \{(C_{k,j,i_1}, C_{k,j,i_2}) \mid C_{k,j,i_1}, C_{k,j,i_2} \in \Sigma \ \wedge \ i_1 = i_2 - 1 \ \wedge \ i_2 \neq j\},$$
$$K = K_1 \cup K_2 \cup K_3 \cup K_4 \cup K_5.$$

Krawędzie dla konkretnego słowa i redukcja przechodnia. Niech $w_{\mathrm{E}} = a_1 a_2 \cdots a_m$ będzie słowem opisującym sekwencyjny przebieg algorytmu (kolejne wykonywane operacje), gdzie $a_t \in \Sigma$ jest symbolem na pozycji t. W grafie Diekerta wierzchołki odpowiadają *wystąpieniom* operacji w słowie, czyli pozycjom:

$$V = \{1, 2, \ldots, m\}.$$

Wprowadzamy funkcję etykietującą (oznaczaną przez λ), która każdej pozycji w słowie przypisuje symbol z alfabetu:

$$\lambda(t) = a_t.$$

$\lambda(t)$ mówi, *jaka operacja* (np. $A_{k,i}$, $B_{k,j,i}$, $C_{k,j,i}$) stoi na wierzchołku t.

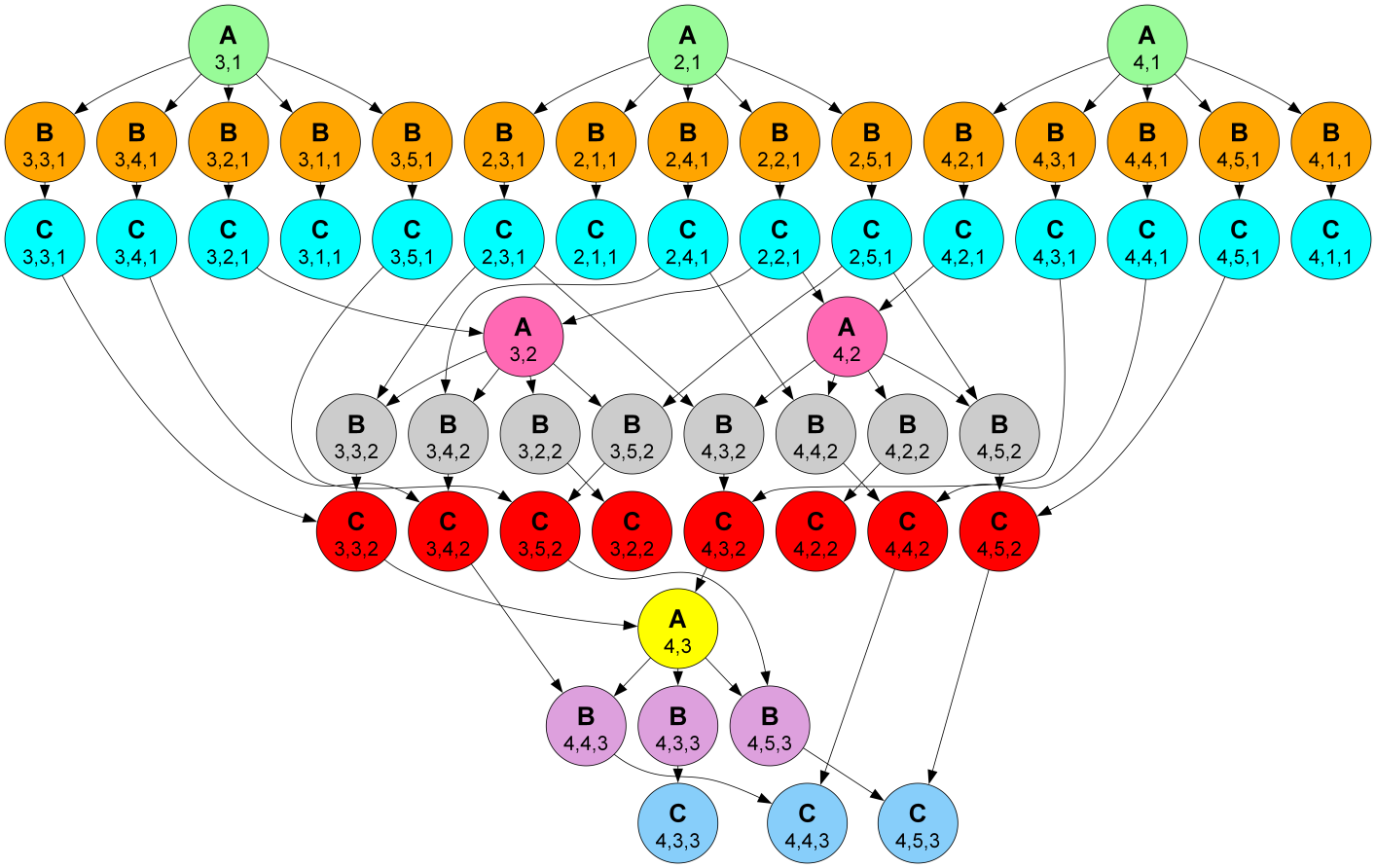
Najpierw budujemy zbiór krawędzi wynikających z konfliktów oraz kolejności w słowie:

$$E(w_{\mathrm{E}}) = \{(t, u) \in V \times V \mid t < u \ \wedge \ (\lambda(t), \lambda(u)) \in K\}.$$

Warunek $t < u$ zapewnia zgodność z porządkiem wystąpień w słowie, natomiast $(\lambda(t), \lambda(u)) \in K$ oznacza, że typy operacji na pozycjach t i u są w konflikcie (zależne).

Zbiór krawędzi grafu Diekerta otrzymujemy przez usuwanie z $E(w_{\mathrm{E}})$ wszystkich krawędzi wynikających *pośrednio* z istnienia ścieżki. W efekcie K zawiera tylko zależności *bezpośrednie*.

Poniżej przedstawiono wygenerowany graf Diekerta dla $N = 4$.



Rysunek 1: Graf zależności Diekerta dla słowa w_{E} (przykład: $N = 4$).

2.5 Postać normalna Foaty

Postać normalna Foaty (FNF) śladu $[w_{\mathrm{E}}]$ można odczytać bezpośrednio z grafu Diekerta. Na rysunku każda warstwa odpowiada kolejnym poziomom/kolorom.

Dla eliminacji słowo w_{E} ma regularną strukturę: dla każdego kroku i najpierw wyznaczamy mnożniki $m_{k,i}$ (operacje A), następnie liczymy iloczyny $n_{k,j,i}$ (operacje B), a na końcu aktualizujemy komórki macierzy (operacje C). Zależności

$A \to B \to C$ oraz fakt, że krok $i+1$ wymaga wyników z kroku i, powodują, że klasy Foaty układają się w powtarzalny wzorzec.

Dla $i = 1, 2, \ldots, n-1$ definiujemy zbiory (dla macierzy rozszerzonej $[A \mid b]$, stąd $j \leq n+1$):

$$FA_i = \{\, A_{k,i} \mid i < k \leq n \,\},$$
$$FB_i = \{\, B_{k,j,i} \mid i < k \leq n \ \wedge\ i \leq j \leq n+1 \,\},$$
$$FC_i = \{\, C_{k,j,i} \mid i < k \leq n \ \wedge\ i \leq j \leq n+1 \,\}.$$

Wówczas postać normalna Foaty dla śladu $[w_{\mathrm{E}}]$ ma postać:

$$\mathrm{FNF}([w_{\mathrm{E}}]) = (FA_1)(FB_1)(FC_1)\,(FA_2)(FB_2)(FC_2)\ \ldots\ (FA_{n-1})(FB_{n-1})(FC_{n-1}).$$

Przykład ($n = 4$). Dla macierzy rozszerzonej 4×5 otrzymujemy:

$$FA_1 = \{A_{2,1}, A_{3,1}, A_{4,1}\},$$
$$FB_1 = \{B_{k,j,1} \mid k \in \{2,3,4\},\ j \in \{1,2,3,4,5\}\},$$
$$FC_1 = \{C_{k,j,1} \mid k \in \{2,3,4\},\ j \in \{1,2,3,4,5\}\},$$
$$FA_2 = \{A_{3,2}, A_{4,2}\}, \quad FB_2 = \{B_{k,j,2} \mid k \in \{3,4\},\ j \in \{2,3,4,5\}\}, \quad FC_2 = \{C_{k,j,2} \mid k \in \{3,4\},\ j \in \{2,3,4,5\}\},$$
$$FA_3 = \{A_{4,3}\}, \quad FB_3 = \{B_{4,j,3} \mid j \in \{3,4,5\}\}, \quad FC_3 = \{C_{4,j,3} \mid j \in \{3,4,5\}\}.$$

Zatem:

$$\mathrm{FNF}([w_{\mathrm{E}}]) = (FA_1)(FB_1)(FC_1)(FA_2)(FB_2)(FC_2)(FA_3)(FB_3)(FC_3),$$

co zgadza się z warstwami widocznymi na wygenerowanym grafie Diekerta.

3 Implementacja

3.1 Opis programu

Do implementacji współbieżnej eliminacji Gaussa-Jordana użyłam języka C oraz biblioteki OpenMP. Program doprowadza macierz rozszerzoną $[A \mid b]$ do postaci $[I \mid x]$ (Gauss-Jordan), gdzie I jest macierzą jednostkową, a x jest wektorem rozwiązań układu $Ax = b$.

Kod programu znajduje się w pliku *program.c*. Do kompilacji wymagane są:

- kompilator `gcc` z obsługą OpenMP,
- biblioteka OpenMP (flaga `-fopenmp`).

3.2 Kompilacja i uruchomienie

Kompilacja:

```
gcc -O2 -fopenmp program.c -o program
```

Uruchomienie:

```
./program input.txt output.txt
```

3.3 Format plików

Wejście zawiera:

1. N (rozmiar macierzy),
2. macierz A ($N \cdot N$ liczb),
3. wektor b (N liczb).

Program buduje macierz rozszerzoną Aug o rozmiarze $N \times (N+1)$. Wyjście zawiera:

1. N,
2. macierz po eliminacji - N wierszy po N liczb,
3. wektor rozwiązań x- 1 wiersz z N liczbami.

3.4 Współbieżność zgodna z klasami Foaty

Dla każdego kroku pivota $i = 0, \ldots, N-1$ program wykonuje trzy fazy, rozdzielone barierami OpenMP, co odpowiada warstwom zależności (klasom Foaty):

- **F1 (normalizacja wiersza pivota):** dzielenie całego wiersza i przez pivot $Aug_{i,i}$.
- **F2 (wyznaczenie mnożników):** obliczenie $m_k \leftarrow Aug_{k,i}$ dla $k \neq i$.
- **F3 (eliminacja):** dla $k \neq i$ wykonanie $Aug_{k,*} \leftarrow Aug_{k,*} - m_k \cdot Aug_{i,*}$, co zeruje kolumnę i w pozostałych wierszach.

Bariery wymuszają porządek F1→F2→F3, dzięki czemu dane są konsumowane dopiero po ich wytworzeniu (zgodnie z grafem zależności / FNF).

3.5 Pivoting

Przed każdą normalizacją w kroku i wykonywany jest pivoting: wybierany jest wiersz $p \geq i$ o maksymalnej wartości $|Aug_{p,i}|$ i następuje zamiana wierszy i i p. Zapobiega to dzieleniu przez wartości bliskie zeru i zwiększa stabilność numeryczną algorytmu.

3.6 Struktura kodu

W pliku *program.c* znajdują się następujące funkcje:

- *die(msg)* – wypisuje komunikat błędu i kończy program.

```
static void die(const char *msg) {
    fprintf(stderr, "%s\n", msg);
    exit(1);
}
```

- *alloc_matrix(rows, cols)* / *free_matrix(M, rows)* – alokacja i zwalnianie macierzy w układzie per-wiersz (ułatwia zamianę wierszy przy pivotingu).

```
static double **alloc_matrix(int rows, int cols) {
    double **M = (double **)malloc((size_t)rows * sizeof(double *));
    if (!M) die("alloc_matrix: malloc failed");

    for (int i = 0; i < rows; ++i) {
        M[i] = (double *)malloc((size_t)cols * sizeof(double));
        if (!M[i]) die("alloc_matrix: malloc row failed");
    }
    return M;
}

static void free_matrix(double **M, int rows) {
    if (!M) return;
    for (int i = 0; i < rows; ++i) free(M[i]);
    free(M);
}
```

- *read_input(in, &N, &Aug)* – wczytuje dane i buduje macierz rozszerzoną $[A \mid b]$.

```
static void read_input(FILE *in, int *N_out, double ***Aug_out) {
    int N;
    if (fscanf(in, "%d", &N) != 1) die("Blad: brak N na wejsciu.");
    if (N <= 0) die("Blad: N <= 0.");

    // tworzymy macierz rozszerzoną
    double **Aug = alloc_matrix(N, N + 1);

    // Wczytujemy A: N*N liczb do kolumn 0..N-1
```

```
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j)
            if (fscanf(in, "%lf", &Aug[i][j]) != 1)
                die("Blad wczytywania macierzy A.");

    // Wczytujemy b: N liczb do ostatniej kolumny (indeks N)
    for (int i = 0; i < N; ++i)
        if (fscanf(in, "%lf", &Aug[i][N]) != 1)
            die("Blad wczytywania wektora b.");

    // wyniki
    *N_out = N;
    *Aug_out = Aug;
}
```

- *gauss_jordan(N, Aug)* – Gauss-Jordan z pivotingiem, zrealizowany w trzech fazach z barierami OpenMP.

```
static void gauss_jordan(int N, double **Aug) {

    const double EPS_PIVOT = 1e-12; //próg dla "zbyt małego" pivota; jeśli |pivot|
        < EPS_PIVOT, traktujemy pivot jako 0

    double *m = (double *)malloc((size_t)N * sizeof(double)); // mnożniki
    if (!m) die("malloc failed for multipliers");

    // Iterujemy po kolejnych kolumnach / pivotach i = 0..N-1
    for (int i = 0; i < N; ++i) {

        int p = i;
        double best = fabs(Aug[i][i]);
        // Szukamy najlepszego pivota w kolumnie i
        for (int r = i + 1; r < N; ++r) {
            double v = fabs(Aug[r][i]);
            if (v > best) { best = v; p = r; }
        }
        if (best < EPS_PIVOT) die("Macierz osobliwa / brak poprawnego pivota.");

        // Zamiana wierszy przez zamianę wskaźników
        if (p != i) {
            double *tmp = Aug[i];
            Aug[i] = Aug[p];
            Aug[p] = tmp;
        }

        double pivot = Aug[i][i];

        // 3 fazy (Foata) z barierami
        #pragma omp parallel
        {
            // F1: normalizacja wiersza pivota
            #pragma omp for schedule(static)
            for (int j = 0; j < N + 1; ++j) {
                Aug[i][j] /= pivot;
            }

            #pragma omp barrier

            // F2: obliczenie mnożników m[k]
            #pragma omp for schedule(static)
```

```
            for (int k = 0; k < N; ++k) {
                m[k] = (k == i) ? 0.0 : Aug[k][i];
            }

            #pragma omp barrier

            // F3: eliminacja wierszy
            #pragma omp for schedule(static)
            for (int k = 0; k < N; ++k) {
                if (k == i) continue;

                double factor = m[k];

                // Jeśli mnożnik jest prawie zerowy, to nic nie zmieniamy
                if (fabs(factor) < EPS_PIVOT) {
                    Aug[k][i] = 0.0;
                    continue;
                }

                // Odejmujemy mnożnik * wiersz pivota od wiersza k
                for (int j = 0; j < N + 1; ++j) {
                    Aug[k][j] -= factor * Aug[i][j];
                }
            }
        }
    }
    // Sprzątanie
    free(m);
}
```

- *fix_neg_zero(x)* – zamienia -0.0 na 0.0, funkcja kosmetyczna.

```
static double fix_neg_zero(double x) {
    if (x == -0.0) return 0.0;
    return x;
}
```

- *write_output(out, N, Aug)* – zapisuje wynik w oczekiwanym formacie.

```
static void write_output(FILE *out, int N, double **Aug) {
    fprintf(out, "%d\n", N);

    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j) {
            if (j) fputc(' ', out);
            fprintf(out, "%.1f", fix_neg_zero(Aug[i][j]));
        }
        fputc('\n', out);
    }

    for (int i = 0; i < N; ++i) {
        if (i) fputc(' ', out);
        fprintf(out, "%.10f", fix_neg_zero(Aug[i][N]));
    }
    fputc('\n', out);
}
```

3.7 Działanie programu

1. Program otwiera plik wejściowy i wczytuje N, macierz A oraz wektor b, tworząc macierz rozszerzoną $[A \mid b]$.
2. Dla każdego $i = 0, \ldots, N-1$ wykonuje pivoting (zamianę wierszy), a następnie trzy równoległe fazy (normalizacja, mnożniki, eliminacja) rozdzielone barierami.
3. Po zakończeniu eliminacji lewa strona jest postaci jednostkowej, a ostatnia kolumna zawiera rozwiązanie x.
4. Program zapisuje wynik do pliku wyjściowego i kończy działanie.

```
int main(int argc, char **argv) {
    if (argc != 3) {
        fprintf(stderr, "Uzycie: %s input.txt output.txt\n", argv[0]);
        return 1;
    }

    // Otwórz plik wejściowy
    FILE *in = fopen(argv[1], "r");
    if (!in) die("Nie mozna otworzyc pliku wejsciowego.");

    // Wczytaj dane
    int N = 0;
    double **Aug = NULL;
    read_input(in, &N, &Aug);
    fclose(in);

    // Wykonaj eliminację współbieżną
    gauss_jordan(N, Aug);

    // Zapisz wynik
    FILE *out = fopen(argv[2], "w");
    if (!out) die("Nie mozna otworzyc pliku wyjsciowego.");
    write_output(out, N, Aug);
    fclose(out);

    // Sprzątanie
    free_matrix(Aug, N);
    return 0;
}
```

3.8 Makefile

Do automatycznej kompilacji dołączony jest plik `Makefile`:

```
CC=gcc
CFLAGS=-O2 -Wall -Wextra -std=c11 -fopenmp
TARGET=program

all: $(TARGET)

$(TARGET): program.c
	$(CC) $(CFLAGS) program.c -o $(TARGET)

clean:
	rm -f $(TARGET)
```

Kompilacja programu:

```
make
```

Uruchomienie programu:

```
./program in.txt out.txt
```

3.9 Przykładowe działanie

3.9.1 Przykład 1

Plik wejściowy `in.txt`:

```
3
2.0 1.0 3.0
4.0 3.0 8.0
6.0 5.0 16.0
6.0 15.0 27.0
```

Uruchomienie:

```
./program in.txt out.txt
```

Plik wyjściowy `out.txt`:

```
3
1.0 0.0 0.0
0.0 1.0 0.0
0.0 0.0 1.0
1.0000000000 1.0000000000 1.0000000000
```

3.9.2 Przykład 2

Plik wejściowy `in2.txt`:

```
15
0.3598381494 0.8409342858 0.2078457000 0.3076592230 0.5746280031 0.3193307736
    0.3614672566 0.7279397259 0.9533114347 0.0125489458 0.3668514230 0.0822051884
    0.6728263458 0.2414697087 0.8907866449
0.0990509280 0.8448001488 0.7335959037 0.4410333892 0.0004641690 0.2571903929
    0.2827449953 0.1344851542 0.8379886466 0.9408241372 0.7638297249 0.7754726642
    0.3612390876 0.4475851775 0.6895253650
0.0611367836 0.5240200388 0.3451626587 0.9028476472 0.9895839527 0.4061565322
    0.5826884804 0.1881432569 0.8365774747 0.2832440536 0.0622547836 0.5527200972
    0.3878068150 0.8118781254 0.9632170592
0.6823218116 0.4663162978 0.8145247654 0.1093526633 0.6058778448 0.1784634465
    0.9460446959 0.9433581349 0.9184359435 0.7168164496 0.6029857273 0.4788331646
    0.0059115238 0.0046892389 0.7971097761
0.9368129044 0.5912644209 0.9445518145 0.3562214748 0.6420745397 0.5789400685
    0.5413422112 0.7382183675 0.1538579404 0.6128888873 0.7117459899 0.9483826822
    0.5363043470 0.9832042774 0.1617978656
0.5352875036 0.1358311358 0.5173959241 0.3176994038 0.4072910492 0.2258247967
    0.8745417488 0.9285354458 0.2594445028 0.4034480317 0.2240388131 0.6246113122
    0.7297402803 0.6790216676 0.6353992729
0.4114219336 0.0139796539 0.7853549691 0.6315148523 0.8068024749 0.5601392847
    0.3525633491 0.2650392302 0.0876001591 0.6359691166 0.2433253886 0.7660147408
    0.9022450545 0.0176539539 0.5241959248
0.2785960288 0.3268870842 0.7827245496 0.2605054719 0.4557001634 0.2953058685
    0.2070036651 0.5655408288 0.7126389005 0.2165884607 0.0632256706 0.1091152343
    0.7542658297 0.5096093700 0.1980688909
0.5963010280 0.5848972130 0.7296366203 0.1246918985 0.5640114466 0.5862422241
    0.9399262573 0.5094661313 0.2112823140 0.4058047461 0.9899567962 0.8544495118
    0.8785187571 0.3366953742 0.4019945394
```

```
0.3812462110 0.2216356655 0.5895190368 0.6389691158 0.1072504053 0.0889349987
    0.3579798574 0.6291786687 0.3718725101 0.6017271235 0.8620720106 0.5571317762
    0.6181986809 0.9641170863 0.6162011510
0.0948736058 0.0646352080 0.0830413107 0.0143082636 0.4444321599 0.0114416364
    0.1665537668 0.9500438661 0.7895594488 0.9186577436 0.0865250001 0.5972672054
    0.0137319643 0.0647808592 0.4413314562
0.6646218739 0.8314134551 0.5496519350 0.9347766200 0.4076652435 0.8008724220
    0.6403601923 0.8993992383 0.4330066054 0.0393853043 0.9755776891 0.5354885381
    0.8611343274 0.8454461847 0.9415865965
0.7115397145 0.7280899996 0.8476419914 0.6123640034 0.9775477666 0.6008731913
    0.7461419270 0.0064400154 0.7794512955 0.2574379281 0.1902886965 0.0548470032
    0.7312017753 0.9393672943 0.8445678922
0.5973289158 0.2003489442 0.1237908044 0.2352716256 0.9280816961 0.3936953738
    0.9808339366 0.5414606953 0.8976471080 0.5914889976 0.6838673279 0.0963694169
    0.3441834018 0.1717851197 0.5220487224
0.9991857941 0.6751889463 0.5910245339 0.3281276648 0.9021790219 0.7498552792
    0.5425374148 0.0792096039 0.7344354286 0.8225782968 0.1015977843 0.6541523947
    0.8510025307 0.1997580345 0.7838164701
0.6384948161 0.8550308482 0.4348680819 0.0930752870 0.7562935026 0.1305411126
    0.3408784191 0.3158685483 0.9779708263 0.9447345766 0.8344664526 0.0308326939
    0.6042533948 0.8313023673 0.6430521759
```

Uruchomienie:

```
./program in2.txt out2.txt
```

Plik wyjściowy `out2.txt`:

```
15
1.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.0 0.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.0 0.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.0 0.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.0 0.0
0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 1.0
−0.5535598625 −0.0506765111 −0.3473165237 −1.6542570703 1.2551143962 −0.3814515523
    −0.9539278208 −0.3477037634 −0.1487458824 0.6463610027 1.1471447557 −0.0669724262
    0.6037270621 0.8729251917 0.6301394487
```

3.10 Weryfikacja rozwiązania sprawdzarką

Program został zweryfikowany przy pomocy checkera z repozytorium `Matrices`.

Wynik działania checkera (brak komunikatu `Error` oraz kod zakończenia 0 oznacza poprawny wynik):

```
2
test.in
test.out
11.646686076594994 2.6782432946422694 0.9236050684430008 −2.0344577974476787
    0.5010134974653657 0.03147027888684877 −3.2764190486543474 −0.5190696446790325
    2.149700623401343 0.18836726014106717 −3.2793228869070523 −1.0596538494446583
    2.0724486398472703 −0.3324013878220072 −0.41176005281973976
```

```
11.6466860766 2.6782432946 0.9236050684 -2.0344577974 0.5010134975 0.0314702789
    -3.2764190487 -0.5190696447 2.1497006234 0.1883672601 -3.2793228869 -1.0596538494
    2.0724486398 -0.3324013878 -0.4117600528

Process finished with exit code 0
```